

A Console-Based Customer Support Ticket Management System in C: Design, Implementation, and Analysis

R. Rajesh

Sec: 6 | PRN: 250200495

App No: SCDS/2025/1886

Department of Computer Science and Engineering

K. Venkata Nishanth

Sec: 6 | PRN: 250200134

App No: SCDS/2025/1830

Department of Computer Science and Engineering

Abstract — This paper presents the design and implementation of a console-based Customer Support Ticket Management System developed in the C programming language. The system enables users to create, view, search, and update support tickets using structured data representation via C structs and an in-memory array for storage. The system demonstrates core concepts of procedural programming, struct-based data modeling, and user interaction through standard I/O. The implementation adheres to fundamental software engineering principles including modularity, separation of concerns, and straightforward state management. Performance characteristics, limitations, and potential enhancements are discussed, along with a comparative analysis against modern ticketing architectures. The system serves as an instructive example for teaching data structures and system design at the undergraduate level.

Keywords — C programming, ticket management, struct, console application, procedural design, software engineering

I. INTRODUCTION

Customer support ticket management systems are fundamental tools in service-oriented software environments. They provide a structured mechanism for logging, tracking, and resolving user-reported issues. While modern implementations often leverage databases, web interfaces, and cloud infrastructure, a console-based implementation in C provides significant pedagogical and engineering value by exposing core data structure manipulation, memory management, and modular function design.

This paper describes the design and implementation of a lightweight Customer Support Ticket Management System (CSTMS) written in C. The system operates entirely in memory using a fixed-size array of structs, with user interaction through standard I/O. The primary contributions of this work are: (1) A clean modular C architecture for ticket lifecycle management; (2) Analysis of design tradeoffs in in-memory storage systems; (3) Identification of enhancement pathways for production-grade systems.

II. SYSTEM DESIGN

A. Data Model

The core data model is represented by a C struct named Ticket, which encapsulates all relevant attributes of a support ticket. The struct definition is given below:

```
struct Ticket {
    int id;
    char name[50];
    char issue[100];
    char status[20];
};
struct Ticket tickets[MAX];
int count = 0;
```

The use of fixed-size character arrays (name[50], issue[100], status[20]) simplifies memory management at the cost of flexibility. A global array tickets[MAX] with MAX=100 provides storage for up to 100 concurrent tickets, with a counter variable tracking current usage.

B. Functional Architecture

The system is decomposed into four core functions, following a procedural modular design pattern: createTicket(), viewTickets(), searchTicket(), and updateStatus(). Each function is self-contained and interacts with global state directly, a common pattern in embedded and low-level C applications.

III. IMPLEMENTATION

A. Ticket Creation

The createTicket() function reads user input for ID, name, and issue. The status field is initialized to "Open" by default using strcpy(). The implementation is shown below:

```

void createTicket() {
    printf("Enter Ticket ID: ");
    scanf("%d", &tickets[count].id);
    printf("Enter Customer Name: ");
    scanf(" %[\n]", tickets[count].name);
    printf("Enter Issue: ");
    scanf(" %[\n]", tickets[count].issue);
    strcpy(tickets[count].status, "Open");
    count++;
}

```

The `scanf` format specifier `%[\n]` is used to capture full strings with whitespace, preceded by a space to consume any leftover newline characters from prior input. This is a common and important idiom in C console applications.

B. Ticket Search and Status Update

Linear search is used across the tickets array to locate a record by ID. The `updateStatus()` function follows the same pattern, replacing the status field upon finding a match:

```

void updateStatus() {
    int id, found = 0;
    scanf("%d", &id);
    for (int i = 0; i < count; i++) {
        if (tickets[i].id == id) {
            scanf(" %[\n]",
                tickets[i].status);
            found = 1; break;
        }
    }
}

```

The $O(n)$ linear search is acceptable given the 100-ticket constraint. For larger datasets, a hash map or binary search tree would be appropriate.

IV. ANALYSIS

A. Time Complexity

TABLE I. COMPLEXITY ANALYSIS

Operation	Time	Space	Notes
Create	$O(1)$	$O(1)$	Direct array insertion
View All	$O(n)$	$O(1)$	Full array scan
Search	$O(n)$	$O(1)$	Linear scan by ID
Update	$O(n)$	$O(1)$	Linear scan then write

B. Limitations

The system has several key limitations: (1) data is not persistent across runs; (2) the maximum ticket count is fixed at compile time; (3) no input validation for duplicate IDs; (4) global state reduces testability. These are inherent tradeoffs of the chosen design and are appropriate for educational contexts.

V. CONCLUSION

This paper presented a modular, console-based ticket management system implemented in C. The system effectively demonstrates struct-based data modeling, linear search, and interactive menu design. Future work includes adding file-based persistence, dynamic memory allocation, and a linked-list backend for unbounded ticket storage.

VI. FUTURE WORK

- Add file-based persistence to retain tickets across program runs.
- Replace fixed-size array with a dynamic linked list for unbounded storage.
- Implement input validation to prevent duplicate ticket IDs.
- Introduce priority levels and timestamp fields for richer ticket metadata.
- Develop a web or GUI front-end for improved usability.

REFERENCES

- [1] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.
- [2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley, 1994.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.